

Chapter 7 Study Notes

More SQL: Complex Queries, Triggers, Views & Schema Modification

Fundamentals of Database Systems — Elmasri & Navathe (7th Ed.)

Chapter Overview

This chapter covers advanced SQL features. It is lower-weightage in exams, so focus on understanding core concepts and being able to write queries for the major topics.

Topics covered:

- NULL values and Three-Valued Logic
 - Nested Queries (Subqueries) — IN, ANY, ALL, EXISTS
 - Joined Tables — INNER, OUTER, NATURAL JOIN
 - Aggregate Functions — COUNT, SUM, AVG, MIN, MAX
 - Grouping — GROUP BY and HAVING
 - WITH clause, CASE, and Recursive Queries
 - Assertions and Triggers
 - Views — Creation, Materialization, and Update
 - Schema Modification — ALTER, DROP
-

1. NULL Values and Three-Valued Logic

1.1 Meanings of NULL

A NULL value in SQL can mean three different things:

- Unknown value — *the value exists but is not known (e.g., a person's salary is unknown)*
- Unavailable/withheld value — *the value exists but is being intentionally hidden*
- Not applicable — *the attribute does not apply to this tuple (e.g., Maiden_name for a male employee)*

Important: Each NULL is considered different from every other NULL. You cannot compare NULL = NULL directly.

1.2 Three-Valued Logic

SQL uses three truth values: **TRUE**, **FALSE**, and **UNKNOWN**.

Whenever NULL is involved in a comparison (e.g., NULL = 5), the result is UNKNOWN.

Truth table rules (key ones to remember):

- TRUE AND UNKNOWN = UNKNOWN

- FALSE AND UNKNOWN = FALSE (because FALSE dominates AND)
- TRUE OR UNKNOWN = TRUE (because TRUE dominates OR)
- FALSE OR UNKNOWN = UNKNOWN
- NOT UNKNOWN = UNKNOWN

Only rows where the WHERE condition evaluates to **TRUE** are included in query results. UNKNOWN rows are excluded.

1.3 Checking for NULL: IS NULL / IS NOT NULL

To test if an attribute is NULL, use **IS NULL** or **IS NOT NULL**.

Example — Retrieve names of all employees with no supervisor:

```
SELECT Fname, Lname
FROM   EMPLOYEE
WHERE  Super_ssn IS NULL;
```

☐ **Exam Tip:** Never write WHERE Super_ssn = NULL — this always evaluates to UNKNOWN and returns no rows. Always use IS NULL.

2. Nested Queries (Subqueries)

A nested query is a complete **SELECT-FROM-WHERE** block placed inside the **WHERE** clause of another (outer) query. The inner query is called a **subquery** or **nested query**.

2.1 The IN Operator

Checks whether a value belongs to a set returned by the subquery.

Example — Retrieve project numbers of projects involving Smith as employee or manager:

```
SELECT DISTINCT Pnumber
FROM   PROJECT
WHERE  Pnumber IN
      (SELECT Pnumber
       FROM   PROJECT, DEPARTMENT, EMPLOYEE
       WHERE  Dnum = Dnumber AND Mgr_ssn = Ssn AND Lname = 'Smith')
OR     Pnumber IN
      (SELECT Pno
       FROM   WORKS_ON, EMPLOYEE
       WHERE  Essn = Ssn AND Lname = 'Smith');
```

2.2 ANY / SOME and ALL Operators

These operators compare a single value against a set of values returned by a subquery.

- = ANY (or = SOME) — TRUE if the value equals at least one value in the set (equivalent to IN)
- > ALL — TRUE if the value is greater than every value in the set
- < ALL — TRUE if the value is less than every value in the set

Example — Retrieve employees who earn more than ALL employees in department 5:

```
SELECT Lname, Fname
FROM   EMPLOYEE
WHERE  Salary > ALL (SELECT Salary
                     FROM   EMPLOYEE
                     WHERE  Dno = 5);
```

2.3 Tuple Comparisons in Nested Queries

You can compare tuples of values by placing them in parentheses:

```
SELECT DISTINCT Essn
FROM   WORKS_ON
WHERE  (Pno, Hours) IN (SELECT Pno, Hours
                       FROM   WORKS_ON
                       WHERE  Essn = '123456789');
```

This finds all employees who work the same (project, hours) combinations as employee 123456789.

2.4 Correlated Nested Queries

A **correlated subquery** references an attribute from the outer query. It is re-evaluated once for each tuple processed by the outer query.

Example — Retrieve employees who have a dependent with the same first name and sex:

```
-- Using correlated subquery:

SELECT E.Fname, E.Lname
FROM   EMPLOYEE AS E
WHERE  E.Ssn IN (SELECT Essn
                FROM   DEPENDENT AS D
                WHERE  E.Fname = D.Dependent_name
                AND    E.Sex   = D.Sex);

-- Equivalent JOIN version (Q16A):

SELECT E.Fname, E.Lname
FROM   EMPLOYEE AS E, DEPENDENT AS D
WHERE  E.Ssn = D.Essn
```

```
AND    E.Sex = D.Sex
AND    E.Fname = D.Dependent_name;
```

2.5 EXISTS and NOT EXISTS

EXISTS returns TRUE if the subquery result is non-empty. **NOT EXISTS** returns TRUE if the subquery result is empty. These are Boolean functions.

Example — Retrieve employees who have at least one dependent AND manage a department:

```
SELECT Fname, Lname
FROM   EMPLOYEE
WHERE  EXISTS (SELECT *
               FROM   DEPENDENT
               WHERE  Ssn = Essn)
AND    EXISTS (SELECT *
               FROM   DEPARTMENT
               WHERE  Ssn = Mgr_Ssn);
```

2.6 Universal Quantification with NOT EXISTS (Double Negation)

SQL has no direct "FOR ALL" quantifier. To express "for every X, condition holds," use double negation with NOT EXISTS.

Pattern: *"There does NOT EXIST an X for which the condition does NOT hold."*

Example — Retrieve names of employees who work on ALL projects controlled by department 5:

```
SELECT Fname, Lname
FROM   EMPLOYEE
WHERE  NOT EXISTS
      ((SELECT Pnumber
        FROM   PROJECT
        WHERE  Dno = 5)
      EXCEPT
      (SELECT Pno
        FROM   WORKS_ON
        WHERE  Ssn = Essn));
```

Logic: The inner set difference finds projects (in dept 5) that the employee does NOT work on. NOT EXISTS says: there are no such projects — i.e., the employee works on ALL of them.

❑ **Exam Tip:** The double-negation trick using NOT EXISTS is a classic exam question. Remember: NOT EXISTS (Projects_in_dept5 EXCEPT Projects_worked_on) = works on all projects in dept 5.

2.7 Explicit Sets and Attribute Renaming

You can use a hardcoded set of values directly in the WHERE clause:

```
SELECT DISTINCT Essn
FROM   WORKS_ON
WHERE  Pno IN (1, 2, 3);
```

Use **AS** to rename attributes in query results:

```
SELECT E.Lname AS Employee_name, S.Lname AS Supervisor_name
FROM   EMPLOYEE AS E, EMPLOYEE AS S
WHERE  E.Super_ssn = S.Ssn;
```

3. Joined Tables in SQL

SQL allows join operations to be specified directly in the **FROM** clause, making queries more readable.

3.1 INNER JOIN

Returns only rows where there is a matching row in both tables. This is the **default** join type.

```
SELECT Fname, Lname, Address
FROM   (EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber)
WHERE  Dname = 'Research';
```

3.2 NATURAL JOIN

Performs an equijoin on all attributes with the same name in both tables. No join condition is specified explicitly.

```
SELECT Fname, Lname, Address
FROM   (EMPLOYEE NATURAL JOIN
        (DEPARTMENT AS DEPT(Dname, Dno, Mssn, Msdate)))
WHERE  Dname = 'Research';
```

The department table is renamed so **Dnumber** becomes **Dno**, creating an implicit join on **Dno**.

3.3 OUTER JOINS

Outer joins include tuples that do not have a match in the other table, padding with NULLs.

- **LEFT OUTER JOIN** — all tuples from the left table appear; unmatched right-side attributes are NULL
- **RIGHT OUTER JOIN** — all tuples from the right table appear; unmatched left-side attributes are NULL
- **FULL OUTER JOIN** — combines results of LEFT and RIGHT OUTER JOIN

Example — List every employee with their supervisor (employees with no supervisor still appear):

```
SELECT E.Lname AS Employee_Name, S.Lname AS Supervisor_Name
FROM   EMPLOYEE AS E LEFT OUTER JOIN EMPLOYEE AS S
      ON E.Super_ssn = S.Ssn;
```

Here, the top-level employee (no supervisor) will appear with NULL in the Supervisor_Name column.

3.4 Multiway JOIN

Joins can be nested in the FROM clause to join more than two tables:

```
SELECT Pnumber, Dnum, Lname, Address, Bdate
FROM   ((PROJECT JOIN DEPARTMENT ON Dnum = Dnumber)
        JOIN EMPLOYEE ON Mgr_ssn = Ssn)
WHERE  Plocation = 'Stafford';
```

❑ **Exam Tip:** Know the difference: INNER JOIN excludes non-matching tuples. LEFT OUTER JOIN keeps all left-side tuples. A NATURAL JOIN auto-matches on same-named columns.

4. Aggregate Functions

Aggregate functions collapse many rows into a single summary value. NULL values are **discarded** when aggregate functions are applied.

4.1 Built-in Aggregate Functions

The five standard aggregate functions:

Function	Description
COUNT (*)	Counts all rows in the group (including NULLs)
COUNT(col)	Counts non-NULL values in the column
SUM(col)	Sum of all non-NULL values
AVG(col)	Average of all non-NULL values
MAX(col)	Maximum non-NULL value
MIN(col)	Minimum non-NULL value

Example — Salary statistics for Research department:

```
SELECT SUM(Salary) AS Total_Sal,
       MAX(Salary) AS Highest_Sal,
```

```

MIN(Salary) AS Lowest_Sal,
AVG(Salary) AS Average_Sal
FROM    (EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber)
WHERE   Dname = 'Research';

```

Counting employees:

```

SELECT COUNT(*) FROM EMPLOYEE;           -- all employees
SELECT COUNT(*) FROM EMPLOYEE, DEPARTMENT
WHERE   DNO = DNUMBER AND DNAME = 'Research';  -- Research dept employees

```

4.2 SOME and ALL on Boolean Collections

SOME — returns TRUE if at least one element in a collection is TRUE (like OR across the set).

ALL — returns TRUE only if every element in the collection is TRUE (like AND across the set).

5. Grouping: GROUP BY and HAVING

5.1 GROUP BY Clause

GROUP BY partitions the result rows into groups based on one or more attributes. Aggregate functions are then applied to each group independently.

Rule: Every attribute in the SELECT clause must either be in the GROUP BY clause or be inside an aggregate function.

Example — Count employees and average salary per department:

```

SELECT Dno, COUNT(*), AVG(Salary)
FROM    EMPLOYEE
GROUP BY Dno;

```

If Dno has NULL values, SQL creates a separate group for all NULLs.

GROUP BY can be applied to a JOIN result:

```

SELECT Pnumber, Pname, COUNT(*)
FROM    PROJECT, WORKS_ON
WHERE   Pnumber = Pno
GROUP BY Pnumber, Pname;

```

5.2 HAVING Clause

HAVING filters entire groups (after grouping), similar to how WHERE filters individual rows (before grouping).

Example — Show only projects with more than 2 employees:

```

SELECT Pnumber, Pname, COUNT(*)

```

```

FROM    PROJECT, WORKS_ON
WHERE   Pnumber = Pno
GROUP BY Pnumber, Pname
HAVING COUNT(*) > 2;

```

5.3 WHERE vs HAVING — Key Difference

WHERE filters **individual rows** before grouping.

HAVING filters **entire groups** after grouping.

Incorrect query (evaluates HAVING before subquery that uses same aggregation):

```

-- WRONG: This counts only salary>40000 employees per dept,
-- then checks if that count > 5 (not what we want)
SELECT Dno, COUNT(*)
FROM    EMPLOYEE
WHERE   Salary > 40000
GROUP BY Dno
HAVING COUNT(*) > 5;  -- Wrong! Checks count of >40k employees

```

Correct version — for each dept with >5 employees total, count those earning >\$40,000:

```

-- Q28: CORRECT version
SELECT Dnumber, COUNT(*)
FROM    DEPARTMENT, EMPLOYEE
WHERE   Dnumber = Dno
AND     Salary > 40000
AND     Dno IN (SELECT Dno
                FROM    EMPLOYEE
                GROUP BY Dno
                HAVING COUNT(*) > 5)
GROUP BY Dnumber;

```

❑ **Exam Tip:** WHERE filters rows, HAVING filters groups. A common exam trap is mixing them up. The HAVING clause can only reference aggregate functions or GROUP BY attributes.

6. WITH, CASE, and Recursive Queries

6.1 The WITH Clause (Common Table Expressions)

WITH defines a temporary named table used only within the current query. It acts like a one-time view and makes complex queries more readable.

```
-- Q28' using WITH:

WITH BIGDEPTS(Dno) AS
  (SELECT Dno
   FROM   EMPLOYEE
   GROUP BY Dno
   HAVING COUNT(*) > 5)

SELECT Dno, COUNT(*)
FROM   EMPLOYEE
WHERE  Salary > 40000
AND    Dno IN BIGDEPTS
GROUP BY Dno;
```

6.2 The CASE Expression

CASE allows conditional value selection within a query. It can appear anywhere a value is expected (SELECT, WHERE, SET, etc.).

```
-- Give different salary raises per department

UPDATE EMPLOYEE

SET Salary =
  CASE WHEN Dno = 5 THEN Salary + 2000
       WHEN Dno = 4 THEN Salary + 1500
       WHEN Dno = 1 THEN Salary + 3000
       ELSE Salary + 0
  END;
```

6.3 Recursive Queries

Recursive relationships (like employee–supervisor) can be queried recursively using **WITH RECURSIVE**.

Example — Build a table of ALL supervisor–supervisee pairs at any level:

```
WITH RECURSIVE SUP_EMP(SupSsn, EmpSsn) AS
  (SELECT SupervisorSsn, Ssn           -- Base case: direct supervisees
   FROM   EMPLOYEE
   UNION
   SELECT E.Ssn, S.SupSsn              -- Recursive case: go up one more level
   FROM   EMPLOYEE AS E, SUP_EMP AS S
```

```
WHERE E.SupervisorSsn = S.EmpSsn)
```

```
SELECT * FROM SUP_EMP;
```

The query starts with an empty SUP_EMP, builds direct supervisee pairs, then iterates until no new pairs can be added (fixed point).

7. Assertions and Triggers

7.1 CREATE ASSERTION

Assertions specify **general constraints** that go beyond what CHECK constraints and foreign keys can express. They involve multiple tables.

Syntax: `CREATE ASSERTION <name> CHECK (<condition>);`

The condition must NOT be violated — SQL checks it after every relevant modification.

Example — No employee should earn more than their department manager:

```
CREATE ASSERTION SALARY_CONSTRAINT
CHECK (NOT EXISTS
      (SELECT *
       FROM   EMPLOYEE E, EMPLOYEE M, DEPARTMENT D
       WHERE  E.Salary > M.Salary
       AND    E.Dno = D.Dnumber
       AND    D.Mgr_ssn = M.Ssn));
```

7.2 CREATE TRIGGER

Triggers are rules for **active databases**. They automatically execute an action when a specific event occurs and a condition is met.

A trigger has three parts:

- Event — INSERT, UPDATE, DELETE on a table (or specific columns)
- Condition — a WHEN clause that is tested when the event fires
- Action — what to do if the condition is TRUE

Example — Notify supervisor when an employee's new salary exceeds the supervisor's salary:

```
CREATE TRIGGER SALARY_VIOLATION
BEFORE INSERT OR UPDATE OF Salary, Supervisor_ssn ON EMPLOYEE
FOR EACH ROW
WHEN (NEW.SALARY > (SELECT Salary
                   FROM   EMPLOYEE
                   WHERE  Ssn = NEW.Supervisor_Ssn))
INFORM_SUPERVISOR(NEW.Supervisor_Ssn, NEW.Ssn);
```

Key keywords: **BEFORE** / **AFTER**, **FOR EACH ROW**, **NEW** (new value), **OLD** (old value for updates/deletes).

❑ **Exam Tip:** For exams: Know the 3 parts (Event, Condition, Action). Assertions check cross-table constraints proactively; Triggers react to specific events and take action.

8. Views (Virtual Tables)

A **view** is a named query stored in the database. It appears as a virtual table but is not physically stored (unless materialized). Views are always up-to-date.

8.1 Creating Views

```
-- V1: View retaining original column names
CREATE VIEW WORKS_ON1
AS SELECT Fname, Lname, Pname, Hours
FROM EMPLOYEE, PROJECT, WORKS_ON
WHERE Ssn = Essn AND Pno = Pnumber;

-- V2: View with renamed columns
CREATE VIEW DEPT_INFO(Dept_name, No_of_emps, Total_sal)
AS SELECT Dname, COUNT(*), SUM(Salary)
FROM DEPARTMENT, EMPLOYEE
WHERE Dnumber = Dno
GROUP BY Dname;
```

After creation, views can be used like regular tables in queries:

```
SELECT Fname, Lname, Hours
FROM WORKS_ON1
WHERE Pname = 'ProductX';
```

To remove a view: **DROP VIEW WORKS_ON1;**

8.2 View Implementation Strategies

Strategy 1 — **Query Modification:** When a view is queried, the DBMS rewrites it as a query on the base tables on-the-fly. Simple but can be slow for complex view definitions.

Strategy 2 — **View Materialization:** The view result is physically stored as a temporary table when first queried. Requires updating when base tables change.

Materialization update strategies:

- Immediate update — view is refreshed as soon as base tables change
- Lazy update — view is refreshed only when it is next queried

- Periodic update — view is refreshed at scheduled intervals (used in banks, retail); may return stale data

8.3 View Updates

Not all views can be updated. Rules:

- A view on a single table with no aggregation and the primary key preserved → update is possible
- A view with aggregate functions (e.g., COUNT, SUM) → UPDATE is NOT permitted
- A view involving JOINS → often ambiguous; DBMS usually disallows updates

Example of a forbidden view update:

```
UPDATE DEPT_INFO
SET    Total_sal = 100000
WHERE  Dept_name = 'Research';

-- ERROR: Total_sal is computed (SUM), not a base column
```

Use **WITH CHECK OPTION** to ensure updated/inserted rows still satisfy the view's WHERE condition:

```
CREATE VIEW DEPT5EMP AS
    SELECT * FROM EMPLOYEE WHERE Dno = 5
WITH CHECK OPTION;
```

8.4 Views as Authorization Mechanism

Views can be used to restrict data access. A user granted access only to a view cannot see the underlying base table data that is filtered out.

```
-- Allow a user to see only department 5 employees:

CREATE VIEW DEPT5EMP AS
    SELECT * FROM EMPLOYEE WHERE Dno = 5;

-- Then grant access only to this view, not to EMPLOYEE
```

❑ **Exam Tip:** Views are a key security mechanism in SQL. For exams, know when a view CAN and CANNOT be updated, and the difference between query modification and materialization.

9. Schema Modification

The DBA can modify a database schema while it is operational using DDL commands.

9.1 DROP Command

Removes named schema elements (tables, schemas, views, etc.).

Two behaviors:

- CASCADE — also drops all dependent objects (views, constraints referencing this table)
- RESTRICT — fails if any other objects depend on this element

```
DROP SCHEMA COMPANY CASCADE;    -- Drops schema and all its contents
```

```
DROP TABLE EMPLOYEE CASCADE;    -- Drops table and all dependent objects
```

9.2 ALTER TABLE Command

Used to modify an existing table's structure:

- Add or drop a column
- Change a column's data type or default value
- Add or drop constraints

```
-- Add a new column:
```

```
ALTER TABLE EMPLOYEE ADD COLUMN Job VARCHAR(12);
```

```
-- Drop a column (CASCADE removes dependent views):
```

```
ALTER TABLE EMPLOYEE DROP COLUMN Address CASCADE;
```

```
-- Drop a named constraint:
```

```
ALTER TABLE EMPLOYEE DROP CONSTRAINT EMPSUPERFK CASCADE;
```

```
-- Change default value:
```

```
ALTER TABLE DEPARTMENT ALTER COLUMN Mgr_ssn DROP DEFAULT;
```

```
ALTER TABLE DEPARTMENT ALTER COLUMN Mgr_ssn SET DEFAULT '333445555';
```

❏ **Exam Tip:** DROP with CASCADE is destructive and irreversible. RESTRICT is safer. Always know which constraints and views would be affected before using CASCADE.

10. Complete SQL Query Block Structure

The full structure of an SQL SELECT query, with clauses in execution order:

```
SELECT    <attribute and function list>
```

```
FROM      <table list>
```

```
[ WHERE    <condition> ]
```

```
[ GROUP BY <grouping attribute(s)> ]
```

```
[ HAVING    <group condition> ]
```

```
[ ORDER BY <attribute list> ];
```

Execution order: **FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY**

This matters because: HAVING can reference aggregate results (computed after GROUP BY), but WHERE cannot.

11. Quick Reference Summary for Exams

Concept	Key Points
NULL & IS NULL	NULL means unknown/unavailable. Use IS NULL (not = NULL). Three-valued logic: TRUE, FALSE, UNKNOWN.
IN vs EXISTS	IN checks membership in a set. EXISTS checks if subquery returns any rows. NOT EXISTS used for 'for all'.
ANY / ALL	= ANY is same as IN. > ALL means greater than every value in the set.
Correlated subquery	References outer query attribute. Re-executed for each outer tuple. Can usually be rewritten as a JOIN.
Double negation	NOT EXISTS (...EXCEPT...) implements universal quantification ('for all').
INNER JOIN	Only matching rows from both tables. Default join type.
LEFT OUTER JOIN	All rows from left table; NULLs for unmatched right-side columns.
GROUP BY	Groups rows by attribute values. Aggregate functions apply per group.
HAVING	Filters groups (not rows). Applied after GROUP BY. Can use aggregate functions.
WITH clause	Defines a temporary named query result, used within the same query.
CASE	Conditional value expression; can appear in SELECT, WHERE, SET.
ASSERTION	Cross-table constraint checked after any relevant change. Uses NOT EXISTS pattern.
TRIGGER	Event + Condition + Action. Fires automatically on INSERT/UPDATE/DELETE.
CREATE VIEW	Defines virtual table. Always up-to-date. Cannot update aggregate views.
Materialization	Stores view physically. Strategies: immediate, lazy, periodic update.
ALTER TABLE	Add/drop columns, change constraints/defaults on existing tables.
DROP ... CASCADE	Removes object and all dependents. DROP ... RESTRICT fails if dependents exist.